throw and throws in Java

Last Updated : 28 Nov, 2024

In Java, Exception Handling is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Java Exception Handling is a mechanism to handle runtime errors such as NullPointerException, ArrayIndexOutOfBoundsException, etc.

In this article, we will learn about **throw and throws in Java which can handle exceptions in Java.**

Java throw

The throw keyword in Java is used to explicitly throw an exception from a method or any block of code. We can throw either checked or unchecked exception. The throw keyword is mainly used to throw custom exceptions.

Syntax in Java throw

```
throw Instance
```

Example:

```
throw new ArithmeticException(“/ by zero”);
```

But this exception i.e., *Instance* must be of type **Throwable** or a subclass of **Throwable**.

For example, an Exception is a sub-class of Throwable and user-defined exceptions typically extend the Exception class. Unlike C++, data types such as int, char, floats, or non-throwable classes cannot be used as exceptions.

The flow of execution of the program stops immediately after the throw statement is executed and the nearest enclosing **try** block is checked to see if it has a **catch** statement that matches the type of exception. If it finds a match, controlled is transferred to that statement otherwise next enclosing **try** block is checked, and so on. If no matching **catch** is found then the default exception handler will halt the program.

Java throw Examples

Example 1:

Java

```java
// Java program that demonstrates the use of throw
class ThrowExcep {
    static void fun()
    {
        try {
            throw new NullPointerException("demo");
        }
        catch (NullPointerException e) {
            System.out.println("Caught inside fun().");
            throw e; // rethrowing the exception
        }
    }

    public static void main(String args[])
    {
        try {
            fun();
        }
        catch (NullPointerException e) {
            System.out.println("Caught in main.");
        }
    }
}
```

Output

```
Caught inside fun().
Caught in main.
```

Example 2

Java

```java
// Java program that demonstrates
// the use of throw
class Test {
    public static void main(String[] args)
    {
        System.out.println(1 / 0);
    }
}
```

Output

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

The **`throw` and `throws` keywords** play a key role in exception handling in Java. If you want to understand their differences and learn how to implement them correctly, the Java Programming Course offers detailed lessons on exception handling with practical exercises

Java throws

throws is a keyword in Java that is used in the signature of a method to indicate that this method might throw one of the listed type exceptions. The caller to these methods has to handle the exception using a try-catch block.

Syntax of Java throws

> ***type method_name(parameters) throws exception_list***
>
> *exception_list is a comma separated list of all the exceptions which a method might throw.*

In a program, if there is a chance of raising an exception then the compiler always warns us about it and compulsorily we should handle that checked exception, Otherwise, we will get compile time error saying **unreported exception XXX must be caught or declared to be thrown**. To prevent this compile time error we can handle the exception in two ways:

1. By using try catch
2. By using the **throws** keyword

We can use the throws keyword to delegate the responsibility of exception handling to the caller (It may be a method or JVM) then the caller method is responsible to handle that exception.

Java throws Examples

Example 1

Java

```java
// Java program to illustrate error in case
// of unhandled exception
class tst {
    public static void main(String[] args)
    {
        Thread.sleep(10000);
        System.out.println("Hello Geeks");
    }
}
```

Output

```
error: unreported exception InterruptedException; must be caught or
declared to be thrown
```

Explanation

In the above program, we are getting compile time error because there is a chance of exception if the main thread is going to sleep, other threads get the chance to execute the main() method which will cause InterruptedException.

Example 2

Java

```java
// Java program to illustrate throws
class tst {
    public static void main(String[] args)
        throws InterruptedException
    {
        Thread.sleep(10000);
        System.out.println("Hello Geeks");
    }
}
```

Output

```
Hello Geeks
```

Explanation

In the above program, by using the throws keyword we handled the InterruptedException and we will get the output as **Hello Geeks**

Example 3

Java

```java
// Java program to demonstrate working of throws
class ThrowsExecp {

    static void fun() throws IllegalAccessException
    {
        System.out.println("Inside fun(). ");
        throw new IllegalAccessException("demo");
    }
```

```java

    public static void main(String args[])
    {
        try {
            fun();
        }
        catch (IllegalAccessException e) {
            System.out.println("caught in main.");
        }
    }
}
```

Output

```
Inside fun().
caught in main.
```

Important Points to Remember about throws Keyword

- throws keyword is required only for checked exceptions and usage of the throws keyword for unchecked exceptions is meaningless.
- throws keyword is required only to convince the compiler and usage of the throws keyword does not prevent abnormal termination of the program.
- With the help of the throws keyword, we can provide information to the caller of the method about the exception.

Next Article >

User-defined Custom Exception in Java

Similar Reads

Java Tutorial

Java is one of the most popular and widely used programming languages which is used to develop mobile apps, desktop apps, web apps, web servers, games, and enterprise-level systems. Java was...

4 min read

Java Overview

Java Basics

Java Flow Control

Java Methods

Java Arrays

Java Strings

Java OOPs Concepts

Java Interfaces

Java Collections

Java Exception Handling

Exceptions in Java

Exception Handling in Java is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Java Exception Handling is a mechanism to handle runtime...

10 min read

Checked vs Unchecked Exceptions in Java

In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions. In Java, there are two types ...

5 min read

Java Try Catch Block

In Java exception is an "unwanted or unexpected event", that occurs during the execution of the program. When an exception occurs, the execution of the program gets terminated. To avoid these...

4 min read

final, finally and finalize in Java

This is an important question concerning the interview point of view. final keyword final (lowercase) is a reserved keyword in java. We can't use it as an identifier, as it is reserved. We can use this keyword wit...

13 min read

throw and throws in Java

In Java, Exception Handling is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Java Exception Handling is a mechanism to handle runtime...

4 min read

User-defined Custom Exception in Java

An exception is an issue (run time error) that occurred during the execution of a program. When an exception occurred the program gets terminated abruptly and, the code past the line that generated th...

3 min read

Chained Exceptions in Java

Chained Exceptions in Java allow associating one exception with another, i.e. one exception describes the cause of another exception. For example, consider a situation in which a method throws an...

3 min read

Null Pointer Exception in Java

A NullPointerException in Java is a RuntimeException. In Java, a special null value can be assigned to an object reference. NullPointerException is thrown when a program attempts to use an object reference...

5 min read

Exception Handling with Method Overriding in Java

An Exception is an unwanted or unexpected event, which occurs during the execution of a program i.e at run-time, that disrupts the normal flow of the program's instructions. Exception handling is used to...

6 min read

Java Multithreading

Java File Handling

Java Streams and Lambda Expressions

Java IO

Java Synchronization

Java Regex

Java Networking

JDBC

Java Memory Allocation

Java Interview Questions

Java Practice Problems

Java Projects

Article Tags : Java

Practice Tags : Java